Design Analysis and Algorithm – Lab Work

Week 8

Question 1: HUFFMAN CODING

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_TREE_HT 100
#define MAX_CHAR 256
struct MinHeapNode {
    char data;
    unsigned freq;
    struct MinHeapNode *left, *right;
};
struct MinHeap {
    unsigned size;
    unsigned capacity;
    struct MinHeapNode** array;
};
struct MinHeapNode* newNode(char data, unsigned freq) {
    struct MinHeapNode* temp = (struct MinHeapNode*)malloc(sizeof(struct MinHeapNode));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}
struct MinHeap* createMinHeap(unsigned capacity) {
    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->size = 0;
    minHeap->capacity = capacity;
    minHeap->array = (struct MinHeapNode**)malloc(minHeap->capacity * sizeof(struct MinHeapNode*));
    return minHeap;
}
void swapMinHeapNode(struct MinHeapNode** a, struct MinHeapNode** b) {
    struct MinHeapNode* t = *a;
    *a = *b;
    *b = t;
}
void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->freq < minHeap->array[smallest]->freq)
        smallest = left;

    if (right < minHeap->size && minHeap->array[right]->freq < minHeap->array[smallest]->freq)
        smallest = right;

    if (smallest != idx) {
        swapMinHeapNode(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}
struct MinHeapNode* extractMin(struct MinHeap* minHeap) {
    struct MinHeapNode* temp = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
    --minHeap->size;
    minHeapify(minHeap, 0);
    return temp;
```

```c
}
void insertMinHeap(struct MinHeap* minHeap, struct MinHeapNode* minHeapNode) {
    ++minHeap->size;
    int i = minHeap->size - 1;
    while (i && minHeapNode->freq < minHeap->array[(i - 1) / 2]->freq) {
        minHeap->array[i] = minHeap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    minHeap->array[i] = minHeapNode;
}
void buildMinHeap(struct MinHeap* minHeap) {
    int n = minHeap->size - 1;
    for (int i = (n - 1) / 2; i >= 0; --i)
        minHeapify(minHeap, i);
}
int isLeaf(struct MinHeapNode* root) {
    return !(root->left) && !(root->right);
}
void printCodes(struct MinHeapNode* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }
    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }
```

```c
    if (isLeaf(root)) {
        printf("  %c   |   %3d   | ", root->data, root->freq);
        for (int i = 0; i < top; ++i)
            printf("%d", arr[i]);
        printf("\n");
    }
}
void calculateSpaceSaving(struct MinHeapNode* root, int arr[], int top, int* totalOriginalBits, int* totalCompressedBi
    if (root->left) {
        arr[top] = 0;
        calculateSpaceSaving(root->left, arr, top + 1, totalOriginalBits, totalCompressedBits);
    }
    if (root->right) {
        arr[top] = 1;
        calculateSpaceSaving(root->right, arr, top + 1, totalOriginalBits, totalCompressedBits);
    }
    if (isLeaf(root)) {
        *totalOriginalBits += root->freq * 8; // Assuming 8 bits per character
        *totalCompressedBits += root->freq * top;
    }
}
void HuffmanCodes(char data[], int freq[], int size) {
    struct MinHeapNode *left, *right, *top;
    struct MinHeap* minHeap = createMinHeap(size);
    for (int i = 0; i < size; ++i)
        minHeap->array[i] = newNode(data[i], freq[i]);
    minHeap->size = size;
    buildMinHeap(minHeap);
```

```c
    while (minHeap->size != 1) {
        left = extractMin(minHeap);
        right = extractMin(minHeap);
        top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        insertMinHeap(minHeap, top);
    }
    int arr[MAX_TREE_HT], topIdx = 0;
    printf("\nChar | Freq | Huffman Code\n");
    printf("--------------------------\n");
    struct MinHeapNode* root = extractMin(minHeap);
    printCodes(root, arr, topIdx);
    int totalOriginalBits = 0, totalCompressedBits = 0;
    calculateSpaceSaving(root, arr, 0, &totalOriginalBits, &totalCompressedBits);
    printf("\nSpace Saving Analysis:\n");
    printf("----------------------\n");
    printf("Original size (8 bits per character): %d bits\n", totalOriginalBits);
    printf("Compressed size (Huffman coding): %d bits\n", totalCompressedBits);
    printf("Space saved: %d bits (%.2f%%)\n",
           totalOriginalBits - totalCompressedBits,
           ((float)(totalOriginalBits - totalCompressedBits) / totalOriginalBits) * 100);
}
```

```c
int main() {
    printf("CH.SC.U4CSE24137\n");
    char str[1000];
    int freq[MAX_CHAR] = {0};
    printf("Enter the string: ");
    scanf("%s", str);
    for (int i = 0; str[i] != '\0'; i++) {
        freq[(unsigned char)str[i]]++;
    }
    char unique_chars[MAX_CHAR];
    int unique_freqs[MAX_CHAR];
    int n = 0;
    for (int i = 0; i < MAX_CHAR; i++) {
        if (freq[i] > 0) {
            unique_chars[n] = (char)i;
            unique_freqs[n] = freq[i];
            n++;
        }
    }
    HuffmanCodes(unique_chars, unique_freqs, n);
    return 0;
}
```

Output:

```
CH.SC.U4CSE24137
Enter the string: DATAANALYTICSANDINTELLIGENCELABORATORY

Char | Freq | Huffman Code
--------------------------
  N  |    4  | 000
  L  |    4  | 001
  O  |    2  | 0100
  R  |    2  | 0101
  C  |    2  | 0110
  B  |    1  | 01110
  G  |    1  | 01111
  T  |    4  | 100
  Y  |    2  | 1010
  S  |    1  | 10110
  D  |    2  | 10111
  E  |    3  | 1100
  I  |    3  | 1101
  A  |    7  | 111

Space Saving Analysis:
----------------------
Original size (8 bits per character): 304 bits
Compressed size (Huffman coding): 138 bits
Space saved: 166 bits (54.61%)
```

Working:

Huffman Coding.

COMPUTATION AND INTELLIGENCE LABORATORY

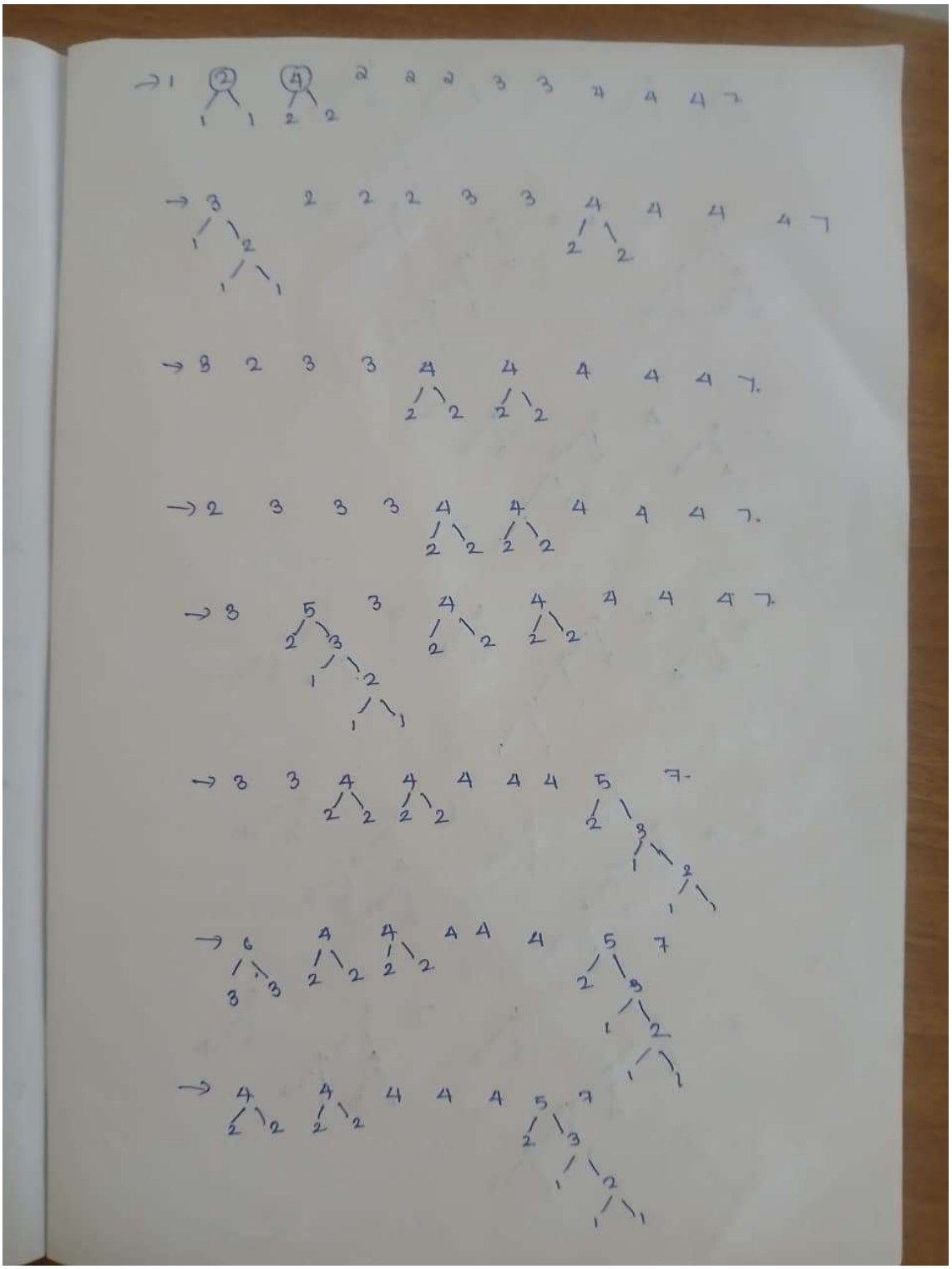
Letter	count
A	7
L	4
N	4
T	4
E	3
I	3
C	2
D	2
O	2
R	2
Y	2
B	1
G	1
S	1

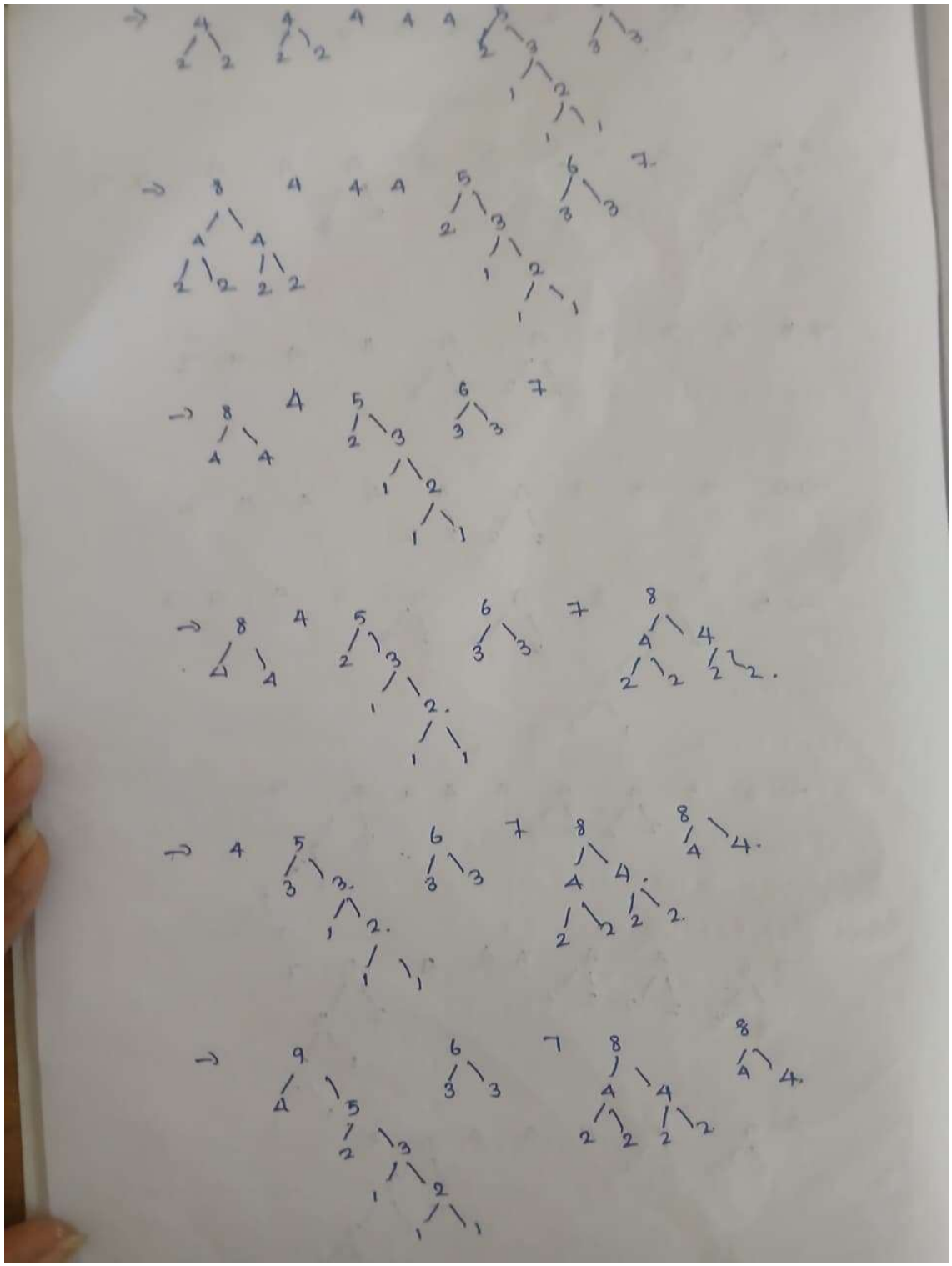
~~Arrange in ascendary.~~

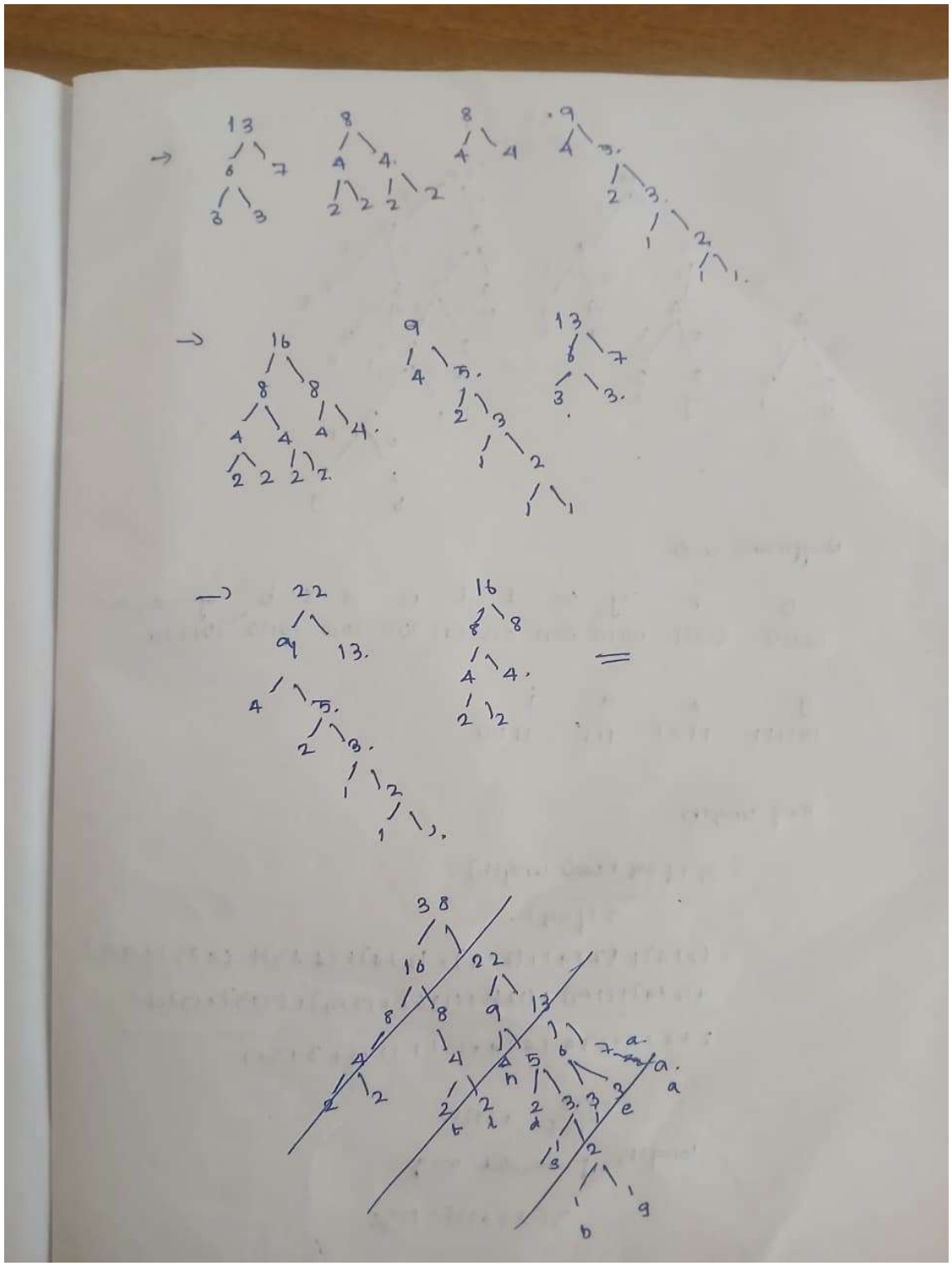
~~B(1) G(1) - S(1) -~~

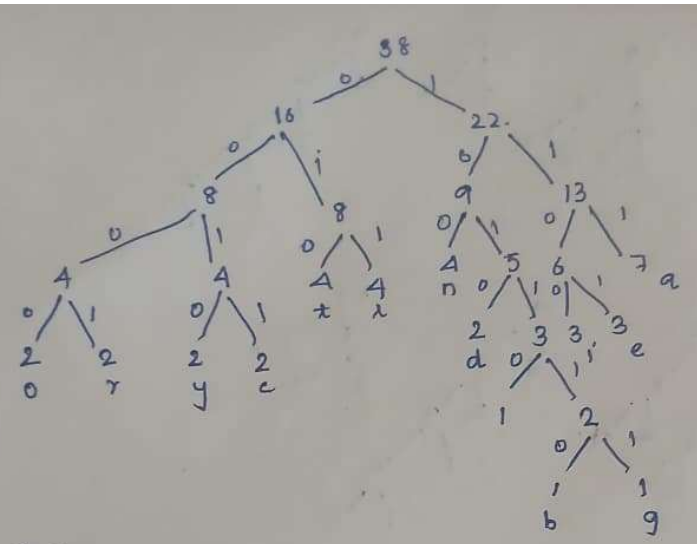
1 1 1 2 2 2 2 2 3 3 4 4 4 7
s b g o r y c d i e t l. n a.

2 1 2 2 2 2 2 3 3 4 4 4 7.
1 1









Huffman code

o	r	y	c	t	l	n	d	s	b	~~g e a i~~
0000	0001	0010	0011	010	011	100	1010	10110	101110	

g	e	a	i
101111	1101	111	1100

Avg length

$$= \frac{\Sigma(\text{freq} \times \text{code length})}{\Sigma(\text{freq})}.$$

$$= \frac{(2\times4)+(2\times4)+(2\times4)+(2\times4)+(4\times3)+(4\times3)+(4\times3)+(2\times4)+(1\times5)+(1\times6)+(1\times6)+(3\times4)+(7\times3)+(3\times4)}{2+2+2+2+4+4+4+2+1+1+1+3+3+7}$$

$$= \frac{138}{138} = 3.6315.$$

length of encoded msg

$$38 \times 3.6315 = 138.$$